
Nonlinear Manifold Reduced Models with Precomputed Weak Operators

Anonymous

Abstract

We study separable learned decoders whose nonlinear latent coordinates admit exact preassembly of the linear and polynomial parts of a weak discrete PDE residual, and we characterise the accuracy, cost and discretisation limits of removing fitted empirical quadrature from the repeated reduced evaluation. **[from generator: abstract endpoints, once the decisive experiments in OUTLINE.md have run]**

1 Introduction

Stub for the build check; see OUTLINE.md.

2 Related work and what is new here

Linear and nonlinear manifold ROMs. Projection-based reduction on a linear subspace [2, 20] is limited on advection-dominated and moving-front problems by the slow decay of the Kolmogorov width [6, 18]. Nonlinear manifold ROMs replace the subspace with the image of a learned decoder: Lee and Carlberg [14] introduced convolutional-autoencoder manifolds with least-squares Petrov–Galerkin projection [4], and Kim et al. [11] made the decoder evaluable on a sample mesh with a shallow masked autoencoder, which is what makes hyper-reduction possible at all on a nonlinear manifold. Our trial manifold (1) is of exactly this family. The decoder we use is a separable one — a coordinate network supplying a spatial bank, composed with a nonlinear coefficient map — rather than a convolutional or masked-dense decoder, and it is this factorisation, not the projection, that we rely on. The projection itself is standard; we claim no novelty for it, and we state the implemented test space, row scaling and stopping rule explicitly (§3.3) because conflating tangent Galerkin with residual least squares was a defect of our own earlier presentation.

Precomputed reduced operators. Caching reduced tensors for polynomial operators is established. Tensorial POD [21] precomputes the reduced coefficients of polynomial nonlinearities so that the online residual and Jacobian are contractions of fixed arrays, and compressive tensor approximations appear already in Carlberg et al. [4]. Quadratic manifolds [1, 8] make the decoder itself polynomial, which yields online-efficient reduced models by the same mechanism; Weder et al. [24] give online-efficient Neural Galerkin models on quadratic manifolds for linear PDEs. Relative to this line, *the tensor construction in (11) is not new*. What differs is the pairing: the coefficient map h_θ is an unrestricted neural network while only the *discrete field operator* is required to be polynomial, so the manifold is not itself quadratic and the guarantees available for quadratic manifolds do not transfer. Whether the extra coefficient nonlinearity is worth its cost at matched reduced dimension is an empirical question, and we treat it as one (§5.1); exact tensor algebra by itself establishes nothing about that. We also note explicitly that the canonical-polyadic decoder of our previous submission admits the same preassembly, so precomputability is not a property of the newer bank: for that architecture one may write $J_u^\top(Au - f) = Dh_\theta^\top[Ch_\theta - d]$ with $C = G^\top AG$ and $d = G^\top(f - Au_g)$. This is an algebraic observation about the printed architecture, not a measured statement about its implementation.

Hyper-reduction. Where the projected operator is not exactly precomputable we use empirical quadrature in the non-negative least-squares form of Hernández et al. [10], with the linear-programming variant of Yano and Patera [25] and energy-based mesh sampling [7] as the nearby alternatives, and DEIM [5] as the interpolation-based one. We use this machinery unchanged. Two things we do differently are worth stating, both of them corrections rather than contributions: the quadrature fits only the term that resists preassembly, since the linear terms are exact by (10) and need no sampling at all; and the node count is a selection among rules fitted offline, not a continuous online control (§3.8). The comparison we care about is therefore between exact preassembly and a fitted rule on the same frozen decoder, holding tests, row scaling, initialisation, time discretisation, stopping and output contract fixed — including the offline cost of each.

Neural operators. FNO [15], DeepONet [16] and the broader operator-learning literature [12] learn a map between function spaces and evaluate it in a single forward pass. They are non-intrusive: they need no discrete operator at inference and apply where no solver exists, which is a capability our method does not have. They also produce one accuracy–cost point per trained model, which is the asymmetry our solver-side controls address (§3.9). We do not treat these as substitutes for one another, and we do not present a head-to-head table as a verdict on either family: we state the condition under which the comparison is meaningful (§3.10), hold the training data and split fixed, and report the result whichever way it falls. In the screens completed so far the neural operator is the more accurate model on the elliptic problem (**from generator: Poisson matched-data screen: per-capacity worst and mean error, ROM and FNO**), and we report that rather than omitting it.

Classical solvers. The separable, constant-coefficient cells admit a direct discrete sine-transform solve [22], which is faster than every reduced model we have measured on those cells; preconditioned Krylov methods [19] are the appropriate iterative comparator, and a reduced model that beats one iterative solver has not thereby beaten the others. We name the direct solver wherever it applies and report it alongside, rather than choosing the comparator that flatters the method. The question a reduced model can be asked on these problems is not whether it beats a fast direct solver on a rectangle — it does not — but what happens where no such solver exists.

What we claim, and what we do not. The contribution we are prepared to defend is a combination and its characterisation: a frozen separable decoder whose spatial bank permits both exact preassembly of the linear and polynomial parts of a weak discrete residual and sampled evaluation of the parts that resist it; a precisely stated weak least-squares solve with recorded stationarity and exit reasons; and measurements of when the compression pays and when it does not, including the cases where it does not. We do not claim a new projection, a new hyper-reduction algorithm, a new tensor construction, a speedup at matched accuracy against a tuned classical solver, dominance over neural operators, or an accuracy–cost frontier. Where the evidence is a single training seed on development cases, we say so next to the number.

3 Method

We reduce a discretised PDE by restricting its state to the image of a frozen learned decoder and solving, in the latent coordinates alone, a least-squares problem built from a small set of smooth weak tests. The construction has four parts: the trial manifold (§3.2), the weak residual and the solver that is actually run (§3.3), the offline assembly of the reduced operators (§3.4), and the three PDE instances (§3.5–§3.7). §3.8 describes empirical quadrature, which is used only for the operator that is not exactly precomputable. §3.9 states precisely which quantities may be varied at deployment, and §3.11 states what the construction does not provide.

3.1 Discrete setting and notation

All problems are posed on $[0, 1]^2$ with homogeneous Dirichlet data and discretised by second-order finite differences on a uniform grid of N intervals per axis. The unknown is the vector of interior nodal values $u \in \mathbb{R}^n$, $n = (N - 1)^2$; boundary values are structurally zero and are not unknowns. We write $A \in \mathbb{R}^{n \times n}$ for the negative discrete Laplacian (five-point stencil, ghost zeros on all four walls) and $f \in \mathbb{R}^n$ for a source or right-hand side. Reduced quantities carry three sizes that must be kept distinct: R is the width of the spatial bank, k the latent dimension solved for online, and M the

number of retained weak tests. Where sampling is used, m is the number of quadrature nodes. Every cell keeps $M > k$, so the latent problem is always overdetermined; the Burgers assembly additionally asserts $M \geq 4k$ and $m \geq 4M$, so its quadrature problem is never under-constrained.

3.2 Trial manifold

The trial manifold is the image of an affine lift composed with a frozen nonlinear coefficient map,

$$u(z) = u_g + G h_\theta(z), \quad G \in \mathbb{R}^{n \times R}, \quad h_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^R, \quad k \leq R \ll n. \quad (1)$$

Both u_g and G are fixed once the mesh is chosen, and θ is frozen for every result reported here. In the implemented cells the Dirichlet data are homogeneous, so $u_g = 0$, and the boundary condition is enforced structurally by a smooth vanishing factor folded into the bank itself:

$$G_{x,:} = \mu(x) g_\phi(x)^\top, \quad \mu(x) = 16 x_1(1 - x_1) x_2(1 - x_2), \quad (2)$$

where $g_\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^R$ is a random-Fourier-feature MLP [23] evaluated pointwise at the nodal coordinates. Because μ vanishes on $\partial[0, 1]^2$, every state in the manifold satisfies the boundary condition exactly and $\partial u / \partial z$ vanishes there as well, for every z and at every resolution.

The coefficient map is a two-layer SiLU MLP plus a linear skip,

$$h_\theta(z) = \varphi_\theta(z) + W^\top z, \quad W \in \mathbb{R}^{k \times R}, \quad (3)$$

so that the decoder Jacobian

$$J_u(z) = \frac{\partial u}{\partial z} = G D h_\theta(z), \quad D h_\theta(z) = D \varphi_\theta(z) + W^\top, \quad (4)$$

retains a latent-independent component. The rank statement this permits is exactly the following, and no more:

$$\text{rank } J_u(z) \leq \min\{k, \text{rank } G\} \leq \min\{k, R\}, \quad (5)$$

with equality to k if and only if $D h_\theta(z)$ has rank k and $\text{range } D h_\theta(z) \cap \ker G = \{0\}$. The linear skip makes the first condition generic but does not prove it: $D \varphi_\theta(z)$ can cancel $W^\top$ at particular z . We therefore measure the rank of the projected Jacobian at every solved query rather than asserting it (§3.3), and report the retained bank rank from the singular values of G .

Two consequences of (1) are used throughout. First, the manifold lies inside the R -dimensional linear span of the bank, so the representation error of the manifold is bounded below by the linear projection error of that span: nonlinearity in h_θ buys a smaller *solved* dimension k , not an escape from the bank's span. Second, all x -dependence factors through G , so restricting the decoder to any point set S is exactly the cached matrix $G_{S,:} \in \mathbb{R}^{|S| \times R}$ times $h_\theta(z)$. This is the property that makes both node sampling (§3.8) and exact operator preassembly (§3.4) available from the same decoder.

3.3 Weak residual, objective, and solver

Tests and row scaling. Let $P \in \mathbb{R}^{M \times n}$ collect M smooth test vectors as rows, together with the discrete integration weight. In every cell the tests are the tensor-product sine vectors $\sin(\pi a x_1) \sin(\pi b x_2)$ sampled at interior nodes, retained for the M smallest eigenvalues λ_i of A , and scaled by a fixed normalisation: the Poisson and Burgers cells use the orthonormal discrete sine basis ($PP^\top = I_M$), while the heat cell uses continuum-normalised tests multiplied by the cell area N^{-2} , i.e. N^{-1} times the orthonormal basis, a uniform scaling chosen so that entries are comparable across meshes. These vectors are exactly the eigenvectors of the five-point operator, which gives the identity we use throughout,

$$PA = \Lambda P, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_M), \quad \lambda_{(a,b)} = 4N^2 \left[\sin^2 \frac{\pi a}{2N} + \sin^2 \frac{\pi b}{2N} \right]. \quad (6)$$

The residual rows are additionally scaled by a diagonal $\Lambda_\star^{-1}$ whose choice is stated per PDE below; the scaling is part of the objective, not a cosmetic normalisation, and it changes which solution the least-squares problem selects.

The objective that is minimised. Given a fully discrete residual $r(\cdot)$ for the PDE at hand, the reduced problem solved online is the row-scaled, weak-tested least-squares problem

$$z^* = \arg \min_{z \in \mathbb{R}^k} \frac{1}{2} \|r_w(z)\|_2^2, \quad r_w(z) = \Lambda_*^{-1} P r(u(z)) \in \mathbb{R}^M, \quad M > k. \quad (7)$$

This is a least-squares Petrov–Galerkin condition with an explicitly chosen test space and row norm, in the sense of Carlberg et al. [4] and Lee and Carlberg [14]. It is *not* tangent-space Galerkin: the test space is P , fixed and independent of z , not $J_u(z)$; and the system is overdetermined ($M > k$) rather than square.

The update that is taken. Write $J(z) = \partial r_w / \partial z \in \mathbb{R}^{M \times k}$, evaluated by forward-mode automatic differentiation through h_θ [3], $H = J^\top J$ and $g = J^\top r_w$. Each attempt solves the damped normal system

$$(H + \lambda \text{diag}(\text{diag } H)) \delta z = -g, \quad (8)$$

and accepts $z \leftarrow z + \delta z$ only if δz is finite, $\|\delta z\| \leq \Delta$ for a fixed trust radius Δ , and the residual norm strictly decreases. On acceptance $\lambda \leftarrow \max(\lambda/3, 10^{-12})$; on rejection the step is discarded and $\lambda \leftarrow \min(10\lambda, \lambda_{\max})$. This is damped Levenberg–Marquardt with a monotone accept/reject test [17]; there is no line search. The curvature dropped in (8) is exactly $\sum_i (r_w)_i \nabla_z^2 (r_w)_i$. Since every implemented r_w is affine or quadratic in $h_\theta(z)$ and h_θ is the only nonlinearity in z , this dropped term is the residual-weighted curvature of the head, $\sum_i (r_w)_i \sum_a B_{ia} \nabla_z^2 h_{\theta,a}$ in the linear-operator case. We identify it here rather than absorbing it into the name of the method.

Stopping. Three families of exit are recorded and never conflated. The primary one is the scale-free stationarity measure

$$\eta(z, \cdot) = \frac{\|J(z)^\top r_w(z)\|_2}{\|J(z)\|_F \|r_w(z)\|_2}, \quad \text{exit when } \eta(z, \cdot) \leq \eta_{\text{tol}}, \quad (9)$$

with $\|\cdot\|_F$ the Frobenius norm. The second is a residual threshold: relative, $\|r_w(z)\| \leq \tau \|r_w(z_0)\|$, in the elliptic cell, and absolute, scaled by the norm of the fitted input, in the Burgers time step. The third family collects *stalls*: exhausting the attempt budget $I_{\max}$, a step below $10^{-14}(1 + \|z\|)$, the damping reaching $\lambda_{\max}$, or a non-finite value. Each solve returns its exit reason, and outputs produced under a stall exit are reported as early-stopped outputs, never as converged solves.

What stationarity does and does not certify. $\eta(z, \cdot) \leq \eta_{\text{tol}}$ certifies that z is a first-order critical point of the specific objective (7) on the frozen manifold. It does not certify a global minimum of that objective; it says nothing about the $n - M$ residual components that no test vector sees; it does not bound the representation error $\min_z \|u(z) - u^*\|$ of the frozen decoder; and it therefore does not bound the physical error of the returned field. We report weak stationarity and physical error as separate quantities throughout, and we record cases in which the weak residual decreases while the field error does not.

Initialisation. No query uses the solution it is predicting. The elliptic cell starts from the cached training code whose stored projected weak prediction is nearest the projected source. The heat cell takes the better of the nearest library code and the mean code after a QR-compressed full-field least-squares fit. The Burgers cell fits the latent code to the supplied initial field on a fixed 48×48 Gauss–Legendre product rule that does not depend on the query mesh: with $Q\mathcal{R}$ the thin QR factorisation of the weight-scaled bank evaluated at the Gauss nodes and $\tilde{u}$ the bilinear interpolation of the dense input at those nodes, the fit minimises $\|\mathcal{R} h_\theta(z) - Q^\top \tilde{u}\|$ by the same LM iteration, started at the training code minimising the corresponding quadratic surrogate. The interpolation of the supplied input is charged to the query.

3.4 Precomputed weak operators

Linear operators. For a fixed linear discrete operator A , substituting (1) into $r(u) = Au - f$ and applying P gives an exactly precomputable reduced form,

$$P(Au(z) - f) = B h_\theta(z) - b, \quad B = PAG \in \mathbb{R}^{M \times R}, \quad b = P(f - Au_g) \in \mathbb{R}^M. \quad (10)$$

In the implemented cells the sine tests diagonalise A , so $B = \Lambda PG$ by (6) and, after row scaling by Λ^{-1} , the reduced matrix is simply PG : a separable sine transform of the bank, assembled offline by two one-dimensional contractions.

Polynomial operators. For a field operator that is polynomial in u with fixed coefficients, the projected term is a fixed multilinear form in h_θ . For the quadratic advection used here, with DG the bank differenced by the fixed stencil,

$$P \left[Gh_\theta \odot (D_x^- + D_y^-) Gh_\theta \right]_i = \sum_{a,b} T_{iab} h_{\theta,a} h_{\theta,b}, \quad T_{iab} = \sum_x P_{ix} G_{xa} (DG)_{xb}, \quad (11)$$

with $DG = D_x^- G + D_y^- G$ and ghost zeros on all four walls. T is not symmetric in (a, b) because its two factors are the bank and the differenced bank, so we store the symmetrised table $Q = T + T^{(a \leftrightarrow b)}$ and evaluate

$$q(h_\theta) = \frac{1}{2} h_\theta^\top Q h_\theta, \quad \frac{\partial q}{\partial h_\theta} = Q h_\theta. \quad (12)$$

Storage is MR^2 entries and one residual evaluation costs $O(MR^2)$; the matrix Qh_θ formed for the residual is reused for the Jacobian, so a residual-and-Jacobian pair costs $O(MR^2 + MRk)$. Bank width therefore enters the online cost quadratically and independently of the latent dimension. The offline build costs $O(nMR^2)$ and is blocked over x so that the $n \times R^2$ intermediate is never materialised.

Exactly when this is available. Three conditions must hold together. (i) The linear operator is fixed, or affinely separable in its parameters, so that B can be assembled once and reused; an arbitrary new coefficient field does not inherit the same offline/online split. (ii) The field operator is polynomial in u with fixed coefficients. (iii) *For the Burgers advection specifically, the sign-dependent upwind switch must be inactive:* the stencil reduces to the fixed backward difference on both axes only where $u > 0$, so (11) is an identity only if the state is non-negative at every stencil node involved. This must hold not only for the data and not only for the converged decoded state, but for *every trial state the solver visits*. Non-negative training data is not sufficient; decoded states can undershoot below zero even when the truth does not, and we report the measured fraction of negative decoded values and the resulting departure from the sign-upwind operator (**[from generator: positivity audit: fraction of negative decoded nodes and tensor-vs-upwind parity per mesh]**) beside every exactness statement. We do not change the full-order discretisation to make the algebra polynomial.

What is and is not mesh-independent. Assembly of G , P , B and T is mesh-dependent and must be redone at each resolution even when the network weights θ transfer unchanged; we charge and report it separately as setup. The cached reduced evaluation – Bh_θ , Qh_θ , the $k \times k$ damped solve (8) – contains no explicit loop over mesh nodes, so its operation count depends on (M, R, k) alone. This does not imply a mesh-independent complete query: reading a dense input and writing a dense requested field must touch $O(n)$ values, and the dense decode $Gh_\theta(z)$ costs $O(nR)$. Nor does it imply mesh-independent iteration counts or mesh-independent accuracy. We measure complete device-query cost with its input, setup, solve and output components reported separately.

3.5 Elliptic instance: Poisson

For $Au = f$ we take $\Lambda_* = \Lambda$, so by (6) and (10) the weak residual is

$$r_w(z) = \Lambda^{-1} P(Au(z) - f) = B_0 h_\theta(z) - b_0, \quad B_0 = PG, \quad b_0 = \Lambda^{-1} Pf. \quad (13)$$

Because the retained tests are orthonormal and $Au^* = f$, this residual is exactly the tested error,

$$r_w(z) = P(u(z) - u^*), \quad (14)$$

so the minimised objective (7) is the squared discrete L^2 error of the reduced state in the retained modes, not merely a residual norm. This is the precise sense in which the elliptic solve minimises an error (energy-type) functional; it is a consequence of the row scaling and holds only for the modes P retains.

Analytically eliminated linear corrections. The deployed elliptic model augments the frozen coefficient vector with a small number c of fixed linear directions $C \in \mathbb{R}^{R \times c}$ and solves

$$\min_{z \in \mathbb{R}^k, y \in \mathbb{R}^c} \|B_0(h_\theta(z) + Cy) - b_0\|_2^2. \quad (15)$$

Let $B_0 C = Q \mathcal{R}$ be a thin QR factorisation. Since Q does not depend on z , the inner minimisation over y has the closed-form solution $\mathcal{R}y = Q^\top (b_0 - B_0 h_\theta(z))$ and the minimised value is

$$\min_y \|B_0(h_\theta(z) + Cy) - b_0\| = \|B_\perp h_\theta(z) - b_\perp\|, \quad B_\perp = (I - QQ^\top)B_0, \quad b_\perp = (I - QQ^\top)b_0. \quad (16)$$

So (15) is solved by running the LM iteration of §3.3 on z alone with the deflated operator, then recovering y with one triangular solve. This is variable projection [9] with a *constant* projector, which matters for the derivative bookkeeping: because $I - QQ^\top$ is independent of z , the elimination introduces no approximation at all, and $J(z) = B_\perp Dh_\theta(z)$ is the *exact* Jacobian of the eliminated residual. The only derivative term dropped anywhere in the elliptic solve is the Gauss–Newton head curvature identified after (8); the y block contributes none, because it enters linearly and is eliminated in closed form. We report both the reduced stationarity of the deflated problem and the augmented stationarity of (15), whose gradient is $[J^\top r; (B_0 C)^\top r]$ normalised by $(\|J\|_F^2 + \|B_0 C\|_F^2)^{1/2} \|r\|$, together with the backward error of the triangular recovery and the numerical rank of $B_0 C$.

3.6 Parabolic instance: heat

The semi-discrete heat equation is $\dot{u} = -\kappa A u$. The implementation advances it with Crank–Nicolson,

$$\left(I + \frac{\Delta t \kappa}{2} A\right) u^{n+1} = \left(I - \frac{\Delta t \kappa}{2} A\right) u^n, \quad (17)$$

and the reduced step is obtained by substituting the manifold into the *fully discrete* equation *before* projecting. With $u^n = u(z_n)$ the fully discrete residual is

$$r_n(z) = \left(I + \frac{\Delta t \kappa}{2} A\right) u(z) - \left(I - \frac{\Delta t \kappa}{2} A\right) u(z_n), \quad (18)$$

which is nonlinear in z because u is. Projecting with P , using (6), and row-scaling by $\Lambda_\star = I + \frac{\Delta t \kappa}{2} \Lambda$ gives the implemented residual

$$r_{w,n}(z) = B_0 h_\theta(z) - D B_0 h_\theta(z_n), \quad D = \text{diag}\left(\frac{1 - \frac{\Delta t \kappa}{2} \lambda_i}{1 + \frac{\Delta t \kappa}{2} \lambda_i}\right), \quad B_0 = P G, \quad (19)$$

and the step is defined as $z_{n+1} = \arg \min_z \|r_{w,n}(z)\|_2$, solved by the LM iteration of §3.3 warm-started at z_n . Two points are structural and were both wrong in the previous presentation. First, the right-hand side of (19) is built from the *decoded current state* $h_\theta(z_n)$ at every step. Carrying instead the quantity that the previous step already made small freezes the recursion: the rollout then reproduces a one-step solution to round-off and the error is silently wrong. Second, z_{n+1} appears through h_θ , so the step is a nonlinear least-squares problem and not a linear solve.

Cholesky and contracted normal equations. Because B_0 and Dh_θ are small, the damped normal matrix in (8) is symmetric positive definite by construction, and the deployed path (`nmrom_cholesky`) factors it by Cholesky and applies two triangular solves instead of a general LU factorisation. A further variant avoids assembling the $M \times k$ weak Jacobian at all: with the fixed Gram matrix $S = B_0^\top B_0 \in \mathbb{R}^{R \times R}$ assembled offline,

$$H = Dh_\theta^\top S Dh_\theta, \quad g = Dh_\theta^\top \left(S(h_\theta(z) - h_\theta(z_0)) + B_0^\top r_{w,n}(z_0) \right), \quad (20)$$

where anchoring the gradient at the initial iterate avoids cancellation near a good warm start. This replaces per-iteration $O(MR)$ work by $O(R^2)$ and is exact; we report it as an algebraic ablation of the same step, not a different model.

3.7 Nonlinear advection: Burgers

The full-order problem is $u_t + u(u_x + u_y) = \nu \Delta u$ with the non-conservative sign-upwind stencil

$$N(u)_{ij} = u_{ij} (\delta_x u + \delta_y u)_{ij}, \quad (\delta_x u)_{ij} = N \cdot \begin{cases} u_{ij} - u_{i-1,j}, & u_{ij} > 0, \\ u_{i+1,j} - u_{ij}, & u_{ij} \leq 0, \end{cases} \quad (21)$$

with δ_y defined analogously on the second index and switching on the *same* centre value u_{ij} , with ghost zeros on all four walls, and backward Euler in time, $r_n(u) = u - u^n + \Delta t(N(u) - \nu \Delta_h u)$. Projecting and row scaling by $\Lambda_\star = I + \Delta t \nu \Lambda$ gives

$$r_{w,n}(z) = (I + \Delta t \nu \Lambda)^{-1} \left[B_0 h_\theta(z) - B_0 h_\theta(z_n) + \Delta t (\hat{N}(z) + \nu \Lambda B_0 h_\theta(z)) \right], \quad (22)$$

where $B_0 = PG$ and the diffusion term is exact by (6). The row scaling is precisely the diagonal of the linearised implicit operator in the modal basis, i.e. the modal form of the Helmholtz preconditioner used by the full-order solver, so the reduced and full-order problems are scaled consistently.

Only the advection projection $\hat{N}(z) \approx PN(Gh_\theta(z))$ requires a choice. Two are implemented and compared:

$$\text{sampler (EQ)} \quad \hat{N}(z) = \sum_{i \in \mathcal{S}} w_i P_{:,i} N_i(Gh_\theta(z)), \quad \text{sign switch retained}, \quad (23)$$

$$\text{precomputed} \quad \hat{N}(z) = \frac{1}{2} h_\theta(z)^\top Q h_\theta(z), \quad \text{exact iff } u \geq 0 \text{ at every stencil node.} \quad (24)$$

The sampled form (23) evaluates the decoder only at the five-point stencil of each retained node, which is the cached $5m \times R$ block times $h_\theta(z)$, and it reproduces the sign-dependent stencil exactly at those nodes. The precomputed form (24) removes sampling entirely but carries the sign restriction of §3.4 and departs from (21) wherever the decoded or trial state undershoots below zero. Time stepping uses a fixed substep count per output interval; each step is warm-started at z_n , and a linear extrapolation $z_n + (z_n - z_{n-1})$ is used instead when, and only when, its residual norm is smaller.

3.8 Empirical quadrature

The sampled rule of (23) is fitted *offline*, from decoder outputs, by non-negative least squares [13] in the empirical-quadrature form of Hernández et al. [10] and Yano and Patera [25]. A candidate node set $\mathcal{C}$ is drawn once with a fixed seed. For each of n_{fit} stored training codes $z^{(\ell)}$ we evaluate the decoded state, form the per-node advection contributions $N_i(Gh_\theta(z^{(\ell)}))$ and the exact full-grid target $PN(Gh_\theta(z^{(\ell)}))$, and stack

$$\min_{w \geq 0} \| \mathcal{G} w - \beta \|_2, \quad \mathcal{G}_{(\ell,i),c} = P_{i,c} N_c(Gh_\theta(z^{(\ell)})), \quad \beta_{(\ell,i)} = [PN(Gh_\theta(z^{(\ell)}))]_i, \quad (25)$$

with the rows normalised by their Euclidean norms before the fit. The support is grown greedily by the Lawson–Hanson criterion with an exact non-negative refit after each addition and a hard cap of m nodes; if the support saturates below m , it is padded with the remaining candidates of largest mean design magnitude and refit, so the achieved node count equals the requested one. Three counts that the previous manuscript did not separate are separated here: the *requested* node count m , the *achieved* support size, and the number of *fit snapshots* n_{fit} used to assemble (25).

A deployed rule is a stored triple: node indices, non-negative weights, and the cached $m \times 5 \times R$ bank block at the five-point stencil of each node. Changing m requires re-solving (25) offline. At deployment one selects among rules that already exist; nothing is re-optimised online. The number of quadrature nodes is therefore a *discrete selection among precomputed rules*, not a continuous accuracy knob.

Two further cautions apply. The NNLS relative fit attained on its own snapshots is recorded, but it is a property of the fitting set and is not a fidelity certificate on unseen states; we certify a rule by measured full-grid parity and by physical error, never by its NNLS residual. And the selected support is one admissible solution among several: rebuilding the same problem under a different floating-point summation order can select different nodes.

What per-node decoder evaluation saves, and what it does not. Because all x -dependence factors through G , evaluating the decoder at the $5m$ stencil coordinates costs $O(mR)$ and is independent of n : this is the property the previous CP spatial head was chosen for, and the separable bank inherits it. It does not reduce the cost of the dense input projection, of the dense output decode $Gh_\theta(z)$ ($O(nR)$), or of the requested full-field output; and it buys nothing for the linear terms, which are already exact and reduced by (10).

3.9 Deployment-time controls at a fixed checkpoint

All controls below act on a single frozen artefact. The network weights θ , the bank G and its width R , the latent dimension k , the test set P and its size M , the time discretisation, and the query contract are identical across every operating point we report. What may change is:

1. the LM attempt budget $I_{\max}$, per solve for the elliptic problem and per time step for the evolution problems;
2. the stopping tolerance — the stationarity threshold η_{tol} of (9), and where applicable the residual threshold, relative (τ) for the elliptic cell and absolute for the Burgers step;
3. the index of the stored empirical-quadrature rule, where sampled evaluation is used at all.

Nothing else is varied, and nothing is re-optimised at query time. An operating point whose solves exited on budget or on a stall is labelled as such and its outputs are reported as early-stopped outputs; losses are reported as losses. We *measure* the resulting error-versus-cost relation. We do not assume it is monotone or that it forms a frontier: a smaller weak residual need not produce a smaller field error, and no amount of additional optimisation can pass the representation limit of the frozen decoder, which we report separately as the best achievable fit of the bank and of the head.

3.10 Intrusive and non-intrusive comparison

The model described here is intrusive. It requires the discrete operator A , the stencil, and the right-hand side at inference, and its offline stage requires assembling B and, where used, T at the deployment resolution. Neural operators such as FNO [15] and DeepONet [16] are non-intrusive: they map data to data and apply where no discretisation or solver is available at all. The two are therefore not substitutes, and a head-to-head accuracy-per-cost table is meaningful only under one stated condition, which we state before any comparison: a discretisation exists and may be used. Under that condition the intrusive route can check and reduce its own equation error, while the non-intrusive route cannot; outside it, the comparison does not apply. We also hold the training data and the train/validation split fixed between the compared models and record the index hashes, so that differences are not attributable to different data.

3.11 What the construction does not provide

We state the boundaries of the method with it, rather than in a closing paragraph.

- *No accuracy certificate.* Weak stationarity (9) is a first-order condition on M tested components of a residual with $n \gg M$ components. It bounds neither the untested components nor the physical error.
- *No escape from the bank.* By (1) the manifold lies in range G ; the bank’s linear projection error is a floor on the achievable field error, whatever the solver does.
- *Offline cost is mesh-dependent.* Frozen weights transfer across meshes, but G , P , B and T do not; rebuilding them is setup work that we charge and report separately.
- *Exact preassembly is conditional.* It requires a fixed or affinely separable linear operator and a polynomial field operator, and for the Burgers upwind stencil it additionally requires the sign restriction of §3.4, which decoded and trial states can violate.
- *Parameter-varying operators need a declared treatment.* A finite affine decomposition can be preassembled term by term; an arbitrary new coefficient field cannot inherit the same split.
- *No convergence theory.* We have no convergence guarantee for the cold-start LM iteration, and no bound on the empirical-quadrature error as a function of m on unseen instances. Both are stated as open.
- *Classical solvers remain strong on the separable cells.* For constant-coefficient Poisson and heat on a rectangle, the discrete sine transform gives a direct solve [22]; we name it as the faster comparator wherever it applies and report it alongside every iterative comparator.

4 Experimental setup

Stub for the build check; see OUTLINE.md.

5 Results

Stub for the build check; see OUTLINE.md.

5.1 Head ablation at matched latent dimension

Stub for the build check.

5.2 Preassembly against fitted quadrature

Stub for the build check.

5.3 Frozen-checkpoint mesh ladder

Stub for the build check.

5.4 Deployment-time controls at a fixed checkpoint

Stub for the build check.

6 Conclusion

Stub for the build check.

References

- [1] Joshua Barnett and Charbel Farhat. Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction. *Journal of Computational Physics*, 464:111348, 2022.
- [2] Peter Benner, Serkan Gugercin, and Karen Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.
- [3] James Bradbury, Roy Frostig, Peter Hawkins, Matthew J. Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. Software.
- [4] Kevin Carlberg, Charbel Bou-Mosleh, and Charbel Farhat. Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86(2):155–181, 2011.
- [5] Saifon Chaturantabut and Danny C. Sorensen. Nonlinear model reduction via discrete empirical interpolation. *SIAM Journal on Scientific Computing*, 32(5):2737–2764, 2010.
- [6] Albert Cohen and Ronald DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 24:1–159, 2015.
- [7] Charbel Farhat, Philip Avery, Todd Chapman, and Julien Cortial. Dimensional reduction of nonlinear finite element dynamic models with finite rotations and energy-based mesh sampling and weighting for computational efficiency. *International Journal for Numerical Methods in Engineering*, 98(9):625–662, 2014.
- [8] Rudy Geelen, Stephen Wright, and Karen Willcox. Operator inference for non-intrusive model reduction with quadratic manifolds. *Computer Methods in Applied Mechanics and Engineering*, 403:115717, 2022.

- [9] Gene H. Golub and Victor Pereyra. The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate. *SIAM Journal on Numerical Analysis*, 10(2): 413–432, 1973.
- [10] J. A. Hernández, M. A. Caicedo, and A. Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.
- [11] Youngkyu Kim, Youngsoo Choi, David Widemann, and Tarek Zohdi. A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder. *Journal of Computational Physics*, 451:110841, 2022.
- [12] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: learning maps between function spaces. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- [13] Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
- [14] Kookjin Lee and Kevin T. Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404: 108973, 2020. Preprint arXiv:1812.08373, 2018.
- [15] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- [16] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- [17] Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.
- [18] Mario Ohlberger and Stephan Rave. Reduced basis methods: success, limitations and future challenges. *Proceedings of ALGORITMY*, pages 1–12, 2016.
- [19] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2 edition, 2003.
- [20] Lawrence Sirovich. Turbulence and the dynamics of coherent structures. I. Coherent structures. *Quarterly of Applied Mathematics*, 45(3):561–571, 1987.
- [21] Răzvan Ștefănescu and Adrian Sandu. Efficient approximation of sparse Jacobians for time-implicit reduced order models. *arXiv preprint arXiv:1402.2018*, 2014. Tensorial POD construction for polynomial reduced operators.
- [22] Paul N. Swarztrauber. The methods of cyclic reduction, Fourier analysis and the FACR algorithm for the discrete solution of Poisson’s equation on a rectangle. *SIAM Review*, 19(3):490–501, 1977.
- [23] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems*, 33, 2020.
- [24] Jules Weder, Paul Schwerdtner, and Benjamin Peherstorfer. Online-efficient model reduction on quadratic manifolds with Neural Galerkin schemes. *arXiv preprint arXiv:2412.17695*, 2024.
- [25] Masayuki Yano and Anthony T. Patera. An LP empirical quadrature procedure for reduced basis treatment of parametrized nonlinear PDEs. *Computer Methods in Applied Mechanics and Engineering*, 344:1104–1123, 2019.